

머신러닝 프로그래밍

Machine Learning Programming

Lecture 5 : 선형회귀 심화



충북대학교
CHUNGBUK NATIONAL UNIVERSITY



충북대학교
산업인공지능연구센터

선형 회귀의 정의 및 해결 방법

↗ 선형 회귀(Linear Regression)란?

데이터들 사이의 직선적인 상관관계를 모델링하여, 입력값(X)에 따른 미래의 결과값(Y)을 예측하는 가장 기초적이고 강력한 기법입니다.

$$y = w_1x + w_0$$

📍 최적의 선을 찾는 두 가지 경로

오차를 최소화하는 가중치(w)를 찾는 방법은 크게 두 가지로 나뉩니다.

1. 최소자승법 (수학적/분석적)

수학 공식을 통해 단 한 번의 행렬 계산으로 정확한 해를 구하는 방법입니다.

2. 경사 하강법 (알고리즘적)

기울기를 따라 조금씩 이동하며 반복적인 학습을 통해 해를 구하는 방법입니다.

최소자승법으로 풀기

- 최소자승법(least square method) : 선형대수의 표기법을 사용하여 방정식으로 선형회귀 문제를 푸는 방법

- 비용함수를 미분

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})^2$$

$$\frac{\partial J}{\partial w_0} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})$$

$$\frac{\partial J}{\partial w_1} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)}) x^{(i)}$$

선형회귀_최소자승법

- 미분한 값을 0으로 만드는 최적값 w_j^* 을 구한다

$$\frac{\partial J}{\partial w_0} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})$$

$$\frac{\partial J}{\partial w_1} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)}) x^{(i)}$$

$$\hat{w}_0 m + \hat{w}_1 \sum x^{(i)} = \sum y^{(i)} \quad (1)$$

$$\hat{w}_0 \sum x^{(i)} + \hat{w}_1 \sum (x^{(i)})^2 = \sum y^{(i)} x^{(i)}$$

선형회귀_최소자승법

- 데이터가 5개만 있다고 가정하고 행렬로 표현

$$X = \begin{bmatrix} 1 & x^{(1)} \\ 1 & x^{(2)} \\ 1 & x^{(3)} \\ 1 & x^{(4)} \\ 1 & x^{(5)} \end{bmatrix} \quad \hat{w} = \begin{bmatrix} \hat{w}_0 \\ \hat{w}_1 \end{bmatrix} \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \\ y^{(5)} \end{bmatrix}$$

$$X^T X = \begin{bmatrix} m & \sum x^{(i)} \\ \sum x^{(i)} & \sum (x^{(i)})^2 \end{bmatrix}$$

- $(X^T X) \hat{w} = X^T y$ 이 (1)번 식과 같음

$$\hat{w} = (X^T X)^{-1} X^T y$$

선형회귀_최소자승법

$$X^T X = \begin{bmatrix} m & \sum x^{(i)} \\ \sum x^{(i)} & \sum (x^{(i)})^2 \end{bmatrix} = \begin{bmatrix} m & m\bar{x} \\ m\bar{x} & \sum (x^{(i)})^2 \end{bmatrix}$$

$$\begin{aligned} |X^T X| &= m \sum (x^{(i)})^2 - (m\bar{x})^2 \\ &= m \left(\sum (x^{(i)})^2 - m\bar{x}^2 \right) \\ &= m \left(\sum (x^{(i)} - \bar{x})^2 \right) \end{aligned}$$

$$\begin{aligned} (X^T X)^{-1} &= \frac{1}{m \sum (x^{(i)} - \bar{x})^2} \begin{bmatrix} \sum (x^{(i)})^2 & -m\bar{x} \\ -m\bar{x} & m \end{bmatrix} \\ &= \frac{1}{\sum (x^{(i)} - \bar{x})^2} \begin{bmatrix} \sum (x^{(i)})^2 / m & -\bar{x} \\ -\bar{x} & 1 \end{bmatrix} \end{aligned}$$

선형회귀_최소자승법

$$\begin{aligned}\hat{w} &= (X^T X)^{-1} X^T y = \begin{bmatrix} \hat{w}_0 \\ \hat{w}_1 \end{bmatrix} \\ &= \frac{1}{\sum (x^{(i)} - \bar{x})^2} \begin{bmatrix} \sum (x^{(i)})^2 / m & -\bar{x} \\ -\bar{x} & m \end{bmatrix} \begin{bmatrix} \sum y^{(i)} \\ \sum x^{(i)} y^{(i)} \end{bmatrix}\end{aligned}$$

$$\hat{w}_1 = \frac{\sum x^{(i)} y^{(i)} - m \bar{x} \bar{y}}{\sum (x^{(i)} - \bar{x})^2}$$

$$\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x}$$

- 변수가 늘어나면 $(X^T X)^{-1}$ 수식만 변경되어 행렬식만 구하면 되어 간단히 연산 가능

최소자승법의 활용

- $X^T X$ 가 존재할 때 문제의 해를 구할 수 있음
- 데이터의 개수가 피쳐의 개수보다 많은 경우가 대부분 이라서 자주 사용됨
- 최소자승법의 장점 : 반복(iteration)과 사용자가 지정하는 하이퍼 매개변수(hyper parameter)가 존재하지 않아서 데이터만 있으면 쉽게 해를 구할 수 있음
- 단점 : 피쳐가 늘어나면 속도가 느려짐
 - 현재 컴퓨터의 연산속도로는 다른 알고리즘에 비해 느린 것이 아님

선형회귀_최소자승법

5-1. ordinary least squares.py

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LinearRegression
6 from sklearn.metrics import mean_squared_error, r2_score
7
8 # 1. 로컬 데이터 불러오기
9 file_path = r'D:\My_project\datasets\Salary_Data.csv'
10 dataset = pd.read_csv(file_path)
11
12 # 데이터 구조 확인 (강의 중 학생들에게 보여주기 좋습니다)
13 print("데이터셋 정보")
14 print(dataset.info())
15 print("\n- 데이터 상위 5행 -")
16 print(dataset.head())
17
18 # 독립 변수(X: 연차)와 종속 변수(y: 연봉) 설정
19 X = dataset.iloc[:, :-1].values
20 y = dataset.iloc[:, -1].values
21
22 # 2. 데이터 분리 (훈련 80%, 테스트 20%)
23 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
24
25 # 3. 모델 생성 및 학습 (내부적으로 최소자승법 OLS 실행)
26 model = LinearRegression()
27 model.fit(X_train, y_train)
28
29 # 4. 결과 예측 및 평가
30 y_pred = model.predict(X_test)
31
32 # 지표 계산
33 rmse = np.sqrt(mean_squared_error(y_test, y_pred))
34 r2 = r2_score(y_test, y_pred)
35
36 print(f"\n--- [실습 결과] ---")
37 print(f"기울기(Weight, w): {model.coef_[0]:.2f}")
38 print(f"절편(Bias, b): {model.intercept_:.2f}")
39 print(f"RMSE: {rmse:.2f}")
40 print(f"R-squared: {r2:.4f}")
41
42 # 5. 시각화: 모델이 찾은 '최적의 직선' 확인
43
44 plt.figure(figsize=(10, 6))
45 plt.scatter(X, y, color='darkorange', label='Actual Data') # 전체 데이터 점
46 plt.plot(X, model.predict(X), color='royalblue', linewidth=2, label='OLS Line') # 회귀선
47 plt.title('Salary vs Experience (OLS Regression)')
48 plt.xlabel('Years of Experience')
49 plt.ylabel('Salary')
50 plt.legend()
51 plt.grid(visible=True, linestyle='--', alpha=0.6)
52 plt.show()
```

선형회귀_최소자승법

5-1. ordinary least squares.py_results

```
기울기(Weight, w): 9312.58
절편(Bias, b): 26780.10
RMSE: 3580.98
R-squared: 0.9882
```



$$\text{Salary} = 9312.58 * \text{YearsExperience} + 26780.10$$

경사하강법의 개념

- 경사하강법(gradient descent) : 경사를 하강하면서 수식을 최소화하는 매개변수의 값을 찾아내는 방법

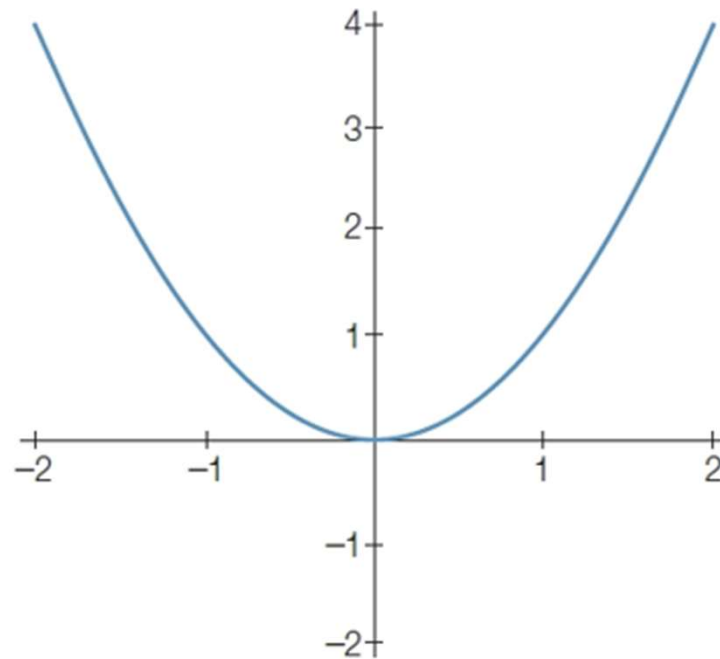


그림1. $y = x^2$ 그래프

선형회귀_경사하강법

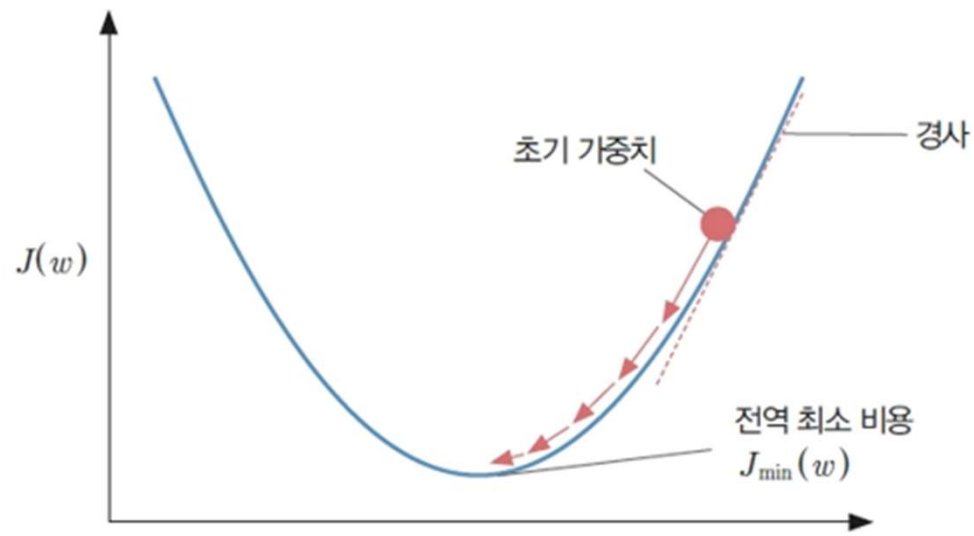


그림2. $y = x^2$ 그래프에 경사하강법 적용

- 점이 최솟값을 달성하는 방향으로 점점 내려감
 - 몇 번 적용할 것인가? : 많이 실행할수록 최솟값에 가까워짐
 - 한 번에 얼마나 많이 내려갈 것인가? : 한 번에 얼마나 많은 공간을 움직일지를 기울기, 즉 경사라고 부름
 - 경사(gradient) : 경사하강법의 하이퍼 매개변수

경사하강법 알고리즘

- $f(x)$ 는 최소화시켜야 하는 값이고,
 $2x$ 는 이를 미분한 값인 경사

$$f(x) = x^2 \rightarrow \frac{dy}{dx} = 2x$$

- 경사하강법의 기본 수식

$$x_{new} = x_{old} - \alpha \times (2x_{old})$$

- x_{old} 는 현재의 x 값, x_{new} 는 경사 값이 적용된 후 생성된 값
- 경사만큼의 변화가 계속 x 에 적용되어 x 의 최솟값 찾음
 - 반복적으로 미분 값을 적용시키면서 더 이상 값이 변하지 않거나 변화가 미미해지는 지점까지 값이 줄어든다

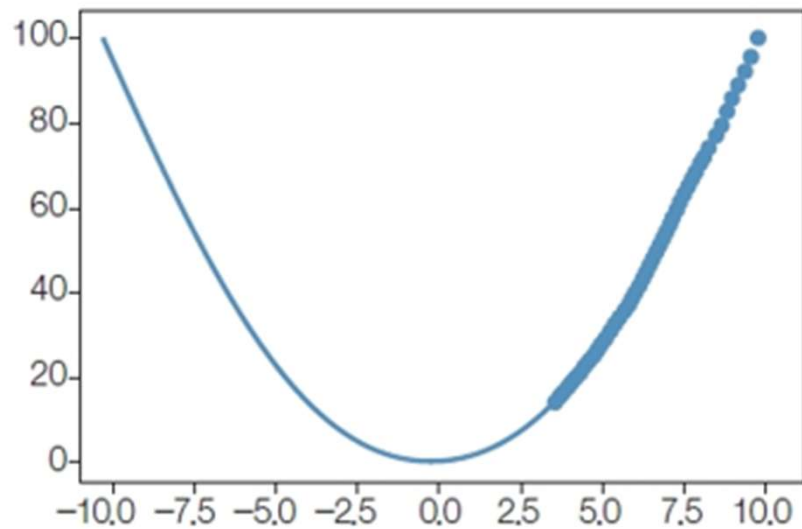
선형회귀_경사하강법

- 경사하강법에서 개발자가 결정해야 할 것
 - 학습률(learning rate)을 얼마나 할 것인가? α 값을 결정

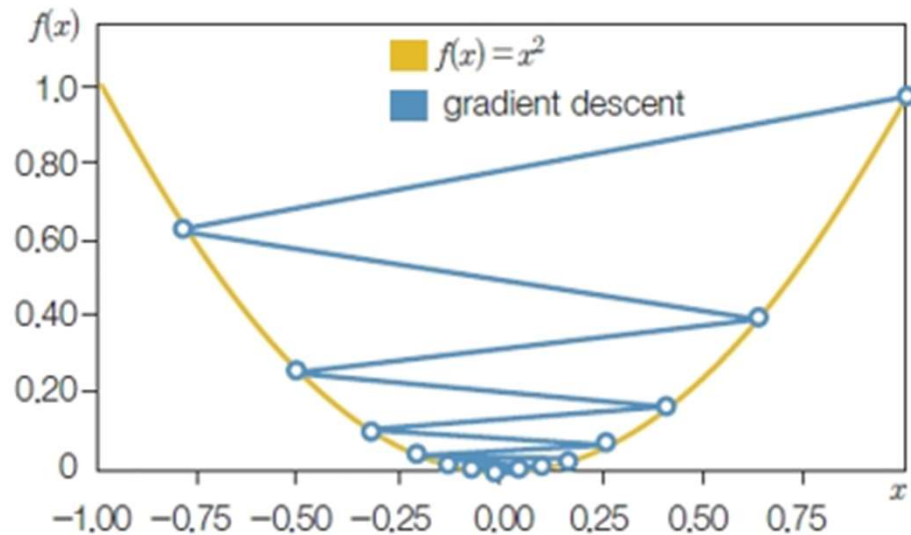
$$x_{new} = x_{old} - \alpha \times (2x_{old})$$

- 반복이 수행될 때마다 최솟값 변화
- 값이 너무 작으면 충분히 많은 반복을 적용해도 원하는 최적값을 찾지 못하는 경우 발생
- 값이 너무 크면 발산하여 최솟값 수렴 않거나 시간이 너무 오래 걸림
- 얼마나 많은 반복(iteration)으로 돌릴 것인가?
 - 반복 횟수가 충분하지 않다면 최솟값을 찾지 못하는 경우 발생
 - 반복 횟수가 너무 많다면 필요 없는 시간을 허비할 수도 있음

선형회귀_경사하강법



(a) 학습률이 작아 최솟값을 수렴하지 못하는 경우



(b) 학습률이 커서 발산한 경우

그림3. 학습률이 너무 크거나 작을때 발생하는 문제

선형회귀_경사하강법

- w 변수가 두 개이기 때문에 3차원 그래프로 J 를 표현

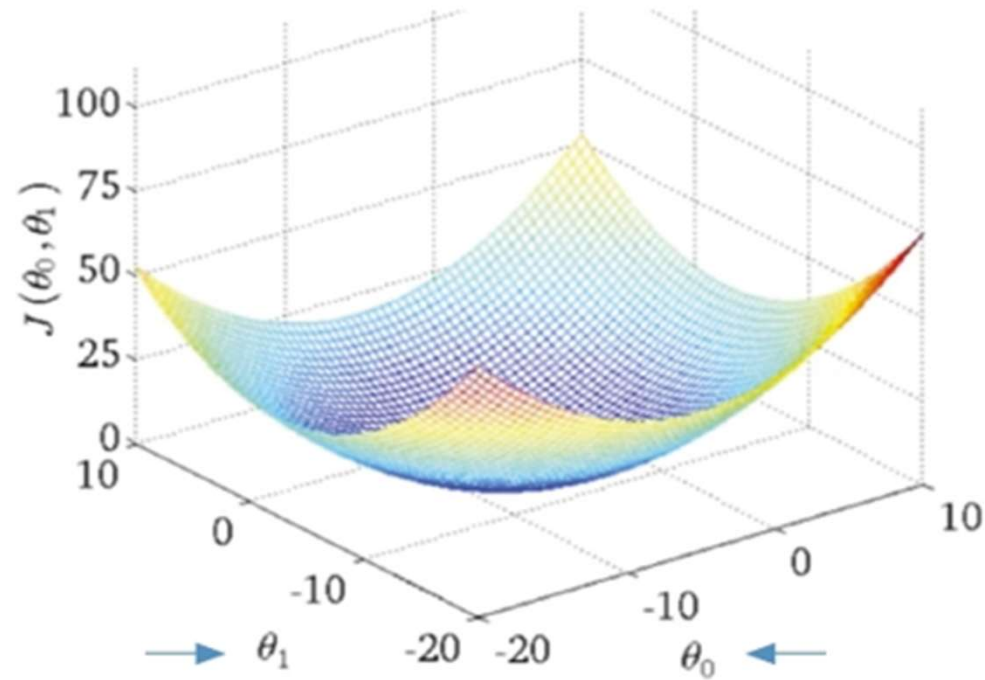


그림5. 비용함수 J 그래프

선형회귀_경사하강법

- 매개변수 w_0 와 w_1 에 대한 업데이트
 - 먼저 임의의 w_0 와 w_1 을 지정
 - 비용함수 J 가 최소화 될 때까지 반복을 수행시켜 w_0 와 w_1 을 학습시킴
 - 비용함수가 더 이상 줄어들지 않거나 정해진 반복이 종료되면 학습이 끝남

loop until convergence {
do $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
}

$$\frac{\partial J}{\partial w_0} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)})$$

$$\frac{\partial J}{\partial w_1} = \frac{1}{m} \sum_{i=1}^m (w_1 x^{(i)} + w_0 - y^{(i)}) x^{(i)}$$

- w_0 와 w_1 업데이트는 동시에(simultaneously) 일어나
하나의 반복 안에서는 서로의 값에 영향을 주지 않음

1. 전체-배치 경사하강법

- 전체-배치 경사하강법(full-batch gradient descent) : 모든 데이터를 한 번에 입력하는 경사하강법

- 배치(batch) : 하나의 데이터셋

- 이전 장에서 배운 경사하강법은 하나의 값에 대한 경사도를 구한 다음 값들을 업데이트

$$x_{new} = x_{old} - \alpha \times (2x_{old})$$

- 실제로는 각 데이터의 경사도를 모두 더해 하나의 값으로 가중치를 업데이트

선형회귀_경사하강법 종류

- 점 한 개씩 사용하여 가중치를 업데이트하지 않는 이유
 - 시간이 오래 걸림
 - 시작점에 따라 지역 최적화(local optimum)에 빠짐 : 그래프 전체에서 최솟점을 찾지 못하고 부분최솟점에 최적점이 위치

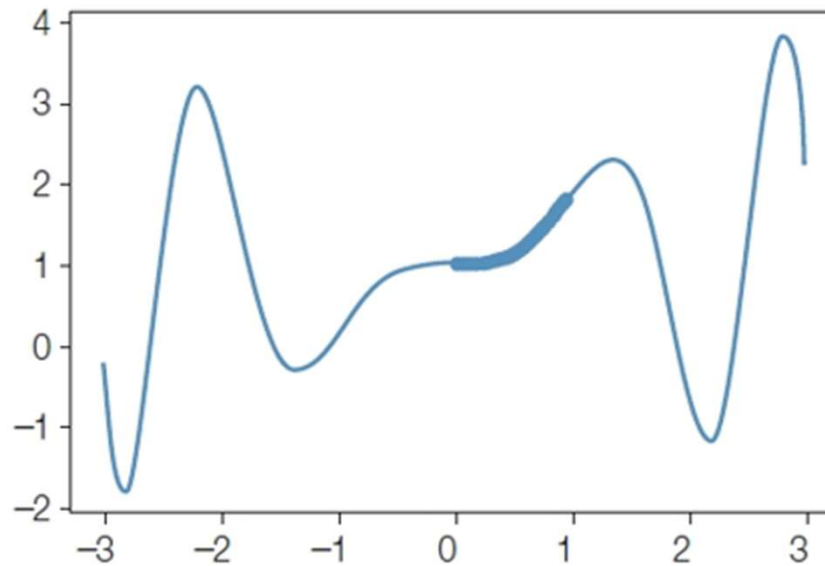


그림6. 점을 한 개씩 사용하여 가중치를 업데이트

선형회귀_경사하강법 종류

- 전체-배치 경사하강법의 특징
 - 업데이트 횟수 감소 : 가중치 업데이트 횟수가 줄어 계산상 효율성 상승
 - 안정적인 비용함수 수렴 : 모든 값의 평균을 구하기 때문에 일반적으로 경사하강법이 갖는 지역 최적화 문제를 만날 가능성도 있음
 - 업데이트 속도 증가 : 대규모 데이터셋을 한 번에 처리하면 모델의 매개변수 업데이트 속도에 문제 발생이 적어짐
 - 데이터가 백만 단위 이상을 넘어가면 하나의 머신에서는 처리가 불가능해져서 메모리 문제가 발생

선형회귀_경사하강법 종류



경사하강법 비교: 점 단위 vs 전체-배치

i 똑같은 수식, 하지만 "데이터를 얼마나 활용하느냐"의 차이



점 단위 경사하강법 (일반/SGD)



데이터 데이터 1개씩만 보고 즉각 판단

업데이트 데이터 1개당 1번 (매우 빈번함)

기울기 현재 위치한 점의 즉각적인 경사도

이동 경로 지그재그로 크게 요동치며 이동 (Noisy)



전체-배치 경사하강법 (Full-batch)



데이터 전체 데이터셋을 한 번에 모두 검토

업데이트 전체를 다 본 후 딱 1번 (매우 신중함)

기울기 모든 데이터 기울기의 총합 (또는 평균)

이동 경로 최적점을 향해 매끄럽고 묵직하게 이동 (Stable)

2. 확률적 경사하강법

- 확률적 경사하강법(Stochastic Gradient Decent, SGD) : 학습용 데이터에서 샘플들을 랜덤하게 뽑아서 사용
- 대상 데이터를 섞은(shuffle) 후, 일반적인 경사하강법 처럼 데이터를 한 개씩 추출하여 가중치 업데이트

선형회귀_경사하강법 종류

■ SGD의 장점

- 빈번한 업데이트를 하기 때문에 데이터 분석가가 모델의 성능 변화를 빠르게 확인
- 데이터의 특성에 따라 훨씬 더 빠르게 결과값을 냄
- 지역 최적화를 회피

■ SGD의 단점

- 대용량 데이터를 사용하는 경우 시간이 매우 오래 걸림
- 결과의 마지막 값을 확인하기 어려움
 - 흔히 '튀는 현상'이라고 불리는데 비용함수의 값이 줄어들지 않고 계속 변화할 때 정확히 언제 루프(loop)가 종료되는지 알 수 없어 판단이 어렵다

3. 미니-배치 경사하강법

- 미니-배치 경사하강법(mini-batch gradient descent) 또는 미니-배치 SGD(mini-batch SGD) : 데이터의 랜덤한 일부분만 입력해서 경사도 평균을 구해 가중치 업데이트
- 에포크(epoch) : 데이터를 한 번에 모두 학습시키는 횟수
 - 전체-배치 SGD를 한 번 학습하는 루프가 실행될 때 1에포크의 데이터가 학습된다고 말함
- 배치 사이즈(batch-size) : 한 번에 학습되는 데이터의 개수
 - 총 데이터가 5012개 있고 배치 사이즈가 512라면 10번의 루프가 돌면서 1에포크를 학습했다고 말함

선형회귀_경사하강법 종류



미니-배치(Mini-batch) 경사하강법

"정확함과 속도 사이의 완벽한 타협점"

✓ 핵심 장점 (Pros)



병렬 연산 최적화 (GPU 가속)

데이터를 뭉치(Batch)로 처리하여 GPU 병렬 연산을 100% 활용, 학습 속도를 획기적으로 향상시킵니다.



부드러운 수렴 (안정성)

개별 데이터의 노이즈에 휘둘리지 않고 평균적인 경사를 따라가므로 SGD보다 안정적으로 하강합니다.



지역 최적화(Local Optimum) 회피

적당한 무작위성(Noise)이 섞여 있어, 좁은 웅덩이를 탈출하여 더 깊은 최적점을 찾을 확률을 높여줍니다.

⚠ 고려사항 (Cons)



하이퍼파라미터 추가

배치 사이즈(32, 128 등)를 얼마로 할지 결정해야 하며, 이 값에 따라 학습 성능이 크게 좌우됩니다.



메모리 자원의 제약

배치 사이즈를 너무 크게 잡으면 GPU의 물리적 메모리 한계를 초과 (Out of Memory)할 수 있습니다.



일반화 성능 민감도

배치 사이즈가 너무 크면 모델이 훈련 데이터에만 과하게 적응하여 실전 데이터에서의 성능이 떨어질 수 있습니다.

선형회귀_경사하강법 종류

"나의 데이터와 컴퓨터 사양에 맞는 최적의 전략은?"

Full-batch GD



전체-배치

"모든 의견을 듣는 신중한 대식가"

데이터 양 **전체 데이터셋**

이동 경로 **매우 안정적 (직선)**

메모리 부담 **매우 높음**

장점: 수렴이 아주 매끄럽고 정확합니다.

단점: 데이터가 크면 컴퓨터가 멈춥니다.

Stochastic GD



확률적 (SGD)

"일단 뛰고 보는 성급한 개인주의자"

데이터 양 **랜덤 샘플 1개**

이동 경로 **매우 불안정 (지그재그)**

메모리 부담 **매우 낮음**

장점: 빠르고 구덩이(Local Opt.) 탈출이 쉽습니다.

단점: 너무 요동쳐서 멈출 때를 모릅니다.

Mini-batch SGD



미니-배치

"장점만 쏙 뽑은 영리한 전략가"

데이터 양 **일부 (32, 64, 128...)**

이동 경로 **안정적 + 적당한 흔들림**

메모리 부담 **효율적 (조절 가능)**

장점: 현대 딥러닝의 표준! GPU를 제일 잘 씁니다.

단점: 배치 사이즈를 정하는 노하우가 필요합니다.

선형회귀_경사하강법

5-2. comparison.py

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import SGDRegressor
4 from sklearn.preprocessing import StandardScaler
5
6 # 1. 데이터 로드 및 표준화
7 df = pd.read_csv(r'D:\My_project\datasets\Salary_Data.csv')
8 X, y = df[['YearsExperience']], df['Salary']
9 scaler = StandardScaler()
10 X_s = scaler.fit_transform(X)
11
12 # 2. 3가지 경사하강법 설정
13 # (1) Stochastic: 1개씩 업데이트
14 sgd_stochastic = SGDRegressor(max_iter=1, learning_rate='constant', eta0=0.01, random_state=42)
15
16 # (2) Mini-batch: 5개씩 묶어서 업데이트
17 sgd_minibatch = SGDRegressor(learning_rate='constant', eta0=0.01, random_state=42)
18
19 # (3) Batch: 30개 전체를 보고 업데이트
20 sgd_batch = SGDRegressor(max_iter=1000, tol=1e-3, learning_rate='constant', eta0=0.01, random_state=42)
21
22 # 3. 시각화 비교
23 plt.figure(figsize=(18, 5))
24
25 # [그래프 1] Stochastic GD
26 plt.subplot(*args: 1, 3, 1)
27 sgd_stochastic.partial_fit(X_s[:1], y[:1].values.ravel())
28 plt.scatter(X_s, y, color='orange', alpha=0.3)
29 plt.plot(*args: X_s, sgd_stochastic.predict(X_s), color='red', label='Stochastic (1 sample)')
30 plt.title(*label: "1. Stochastic GD", fontsize=13, color='red')
31 plt.legend()
32
33 # [그래프 2] Mini-batch GD
34 plt.subplot(*args: 1, 3, 2)
35 sgd_minibatch.partial_fit(X_s[:5], y[:5].values.ravel())
36 plt.scatter(X_s, y, color='orange', alpha=0.3)
37 plt.plot(*args: X_s, sgd_minibatch.predict(X_s), color='blue', label='Mini-batch (5 samples)')
38 plt.title(*label: "2. Mini-batch GD", fontsize=13, color='blue')
39 plt.legend()
40
41 # [그래프 3] Full-Batch GD
42 plt.subplot(*args: 1, 3, 3)
43 sgd_batch.fit(X_s, y)
44 plt.scatter(X_s, y, color='orange', alpha=0.3)
45 plt.plot(*args: X_s, sgd_batch.predict(X_s), color='green', linewidth=3, label='Batch (All 30 samples)')
46 plt.title(*label: "3. Full-Batch GD", fontsize=13, color='green')
47 plt.legend()
48
49 plt.tight_layout()
50 plt.show()
```

선형회귀_최소자승법 vs 경사하강법

5-3. comparison.py

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression, SGDRegressor
4 from sklearn.preprocessing import StandardScaler
5
6 # 1. 데이터 준비
7 df = pd.read_csv(r'D:\My_project\datasets\Salary_Data.csv')
8 X, y = df[['YearsExperience']], df['Salary']
9
10 # 2. OLS (수학적 정답)
11 ols = LinearRegression().fit(X, y)
12
13 # 3. SGD 학습 준비 (표준화 필수)
14 scaler = StandardScaler()
15 X_s = scaler.fit_transform(X)
16
17 # [중간용] 딱 1번만 업데이트한 모델
18 sgd_short = SGDRegressor(max_iter=1, tol=None, eta=0.01, random_state=42).fit(X_s, y)
19 # [오른쪽용] 충분히 학습한 모델
20 sgd_long = SGDRegressor(max_iter=1000, tol=1e-3, eta=0.01, random_state=42).fit(X_s, y)
21
22 # 4. 그래프 출력
23 plt.figure(figsize=(18, 6))
24
25 # [왼쪽] OLS: 수학적 정답
26 plt.subplot(*args: 1, 3, 1)
27 plt.scatter(X, y, color='orange', alpha=0.5)
28 plt.plot(*args: X, ols.predict(X), color='blue', linewidth=3)
29 title_ols = f"OLS: Exact Solution\nw: {ols.coef_[0]:.2f}, b: {ols.intercept_: .2f}"
30 plt.title(title_ols, fontsize=12, fontweight='bold')
31
32 # [중간] SGD (1회 학습): 방향하는 단계
33 plt.subplot(*args: 1, 3, 2)
34 plt.scatter(X_s, y, color='orange', alpha=0.5)
35 plt.plot(*args: X_s, sgd_short.predict(X_s), color='red', linestyle='--', label='1 Iteration')
36 title_short = f"SGD: 1 Iteration\nw: {sgd_short.coef_[0]:.2f}, b: {sgd_short.intercept_[0]:.2f}"
37 plt.title(title_short, fontsize=12, color='red')
38 plt.legend()
39
40 # [오른쪽] SGD (완료): 수렴한 단계
41 plt.subplot(*args: 1, 3, 3)
42 plt.scatter(X_s, y, color='orange', alpha=0.5)
43 plt.plot(*args: X_s, sgd_long.predict(X_s), color='green', linewidth=3, label='1000 Iterations')
44 title_long = f"SGD: Final Convergence\nw: {sgd_long.coef_[0]:.2f}, b: {sgd_long.intercept_[0]:.2f}"
45 plt.title(title_long, fontsize=12, color='green', fontweight='bold')
46 plt.legend()
47 plt.tight_layout()
48 plt.show()
```

선형회귀의 성능 측정 지표

1. MAE

- MAE(Mean Absolute Error) : 평균 절대 잔차
- 모든 테스트 데이터에 대해 예측값과 실제값의 차이에 대해 절대값을 구하고, 이 값을 모두 더한 후에 데이터의 개수만큼 나눈 결과

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| = \frac{1}{n} \sum_{i=1}^n |e_i|$$

- 직관적으로 예측값과 실측값의 차이를 알 수 있음

2. MSE

- MSE(Mean Squared Error) : 평균 제곱 오차
- 모든 테스트 데이터에 대해 예측값과 실제값의 차이에 대해 제곱을 구하고, 이 값을 모두 더한 후에 데이터의 개수만큼 나눈 결과

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n e_i^2$$

- MAE와 달리 오차를 제곱하기 때문에, 큰 오차에 대해 더 큰 패널티를 부여함
- 수학적으로 미분이 가능하여 최적화 과정(최소자승법 등)에서 주로 사용됨

3. RMSE

- RMSE(Root Mean Squared Error) : 평균제곱근 오차
- 오차에 대해 제곱을 한 다음 모든 값을 더하여 평균을 낸 후 제곱근을 구함
- MAE에 비해 상대적으로 값의 차이가 더 큼
- 차이가 크게 나는 값에 대해서 페널티를 주고 싶다면 RMSE 값을 사용

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

4. 결정계수

- 결정계수(R-squared) : 두 개의 값의 증감이 얼마나 일관성을 가지는지 나타내는 지표
- 예측값이 크면 클수록 실제값도 커지고, 예측값이 작으면 실제값도 작아짐
- 두 개의 모델 중 어떤 모델이 조금 더 상관성이 있는지를 나타낼 수 있지만, 값의 차이 정도가 얼마인지는 나타낼 수 없다는 한계가 있음

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \mu)^2}$$

선형회귀 성능 측정 지표 최종 요약

지표 (Metric)	수학적 공식 (Formula)	핵심 특징 (Key Features)	추천 활용 사례 (Best Practice)
MAE 절대 잔차	$MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i$	오차를 있는 그대로 반영. 이상치에 강함 (Robust). 결과가 데이터 단위와 동일.	정직한 해석이 필요할 때: 평균적으로 몇 도(또는 몇 원) 틀리는지 보고할 때.
MSE 제곱 오차	$MSE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i^2$	큰 오차에 제곱 패널티 부여. 수학적 최적화(미분)에 최적화됨.	모델 학습 시: 큰 실수를 방지해야 하는 최적화 과정(비용 함수)으로 사용.
RMSE 제곱근 오차	$RMSE = \sqrt{MSE}$	MSE의 장점(패널티)을 유지하면서 단위를 원래 대로 복구. 해석 가능성 높음.	가장 표준적인 보고 지표: 엄격한 평가와 직관적인 단위가 모두 필요할 때.
R-squared 결정계수	$R^2 = 1 - \frac{RSS}{TSS}$	단위와 상관없이 모델의 설명력(%)을 0~1 사이로 표현. 상대적 성능 비교 용이.	논문 및 타당성 입증: 내 모델이 데이터를 얼마나 잘 설명하는지 비율로 증명할 때.

선형회귀_과대적합과 정규화

1. 과대적합 극복하기

- 편향(bias) : 학습된 모델이 학습 데이터에 대해 만들어진 예측값과 실제값과의 차이
 - 모델의 결과가 얼마나 한쪽으로 쏠려 있는지 나타냄
 - 편향이 크면 과소적합이 많음
- 분산(variance) : 학습된 모델이 테스트 데이터에 대해 만들어진 예측값과 실제값과의 차이
 - 모델의 결과가 얼마나 퍼져 있는지 나타냄
- 편향-분산 트레이드오프(bias-variance trade-off) : 편향과 분산의 상충관계

선형회귀_과대적합과 정규화

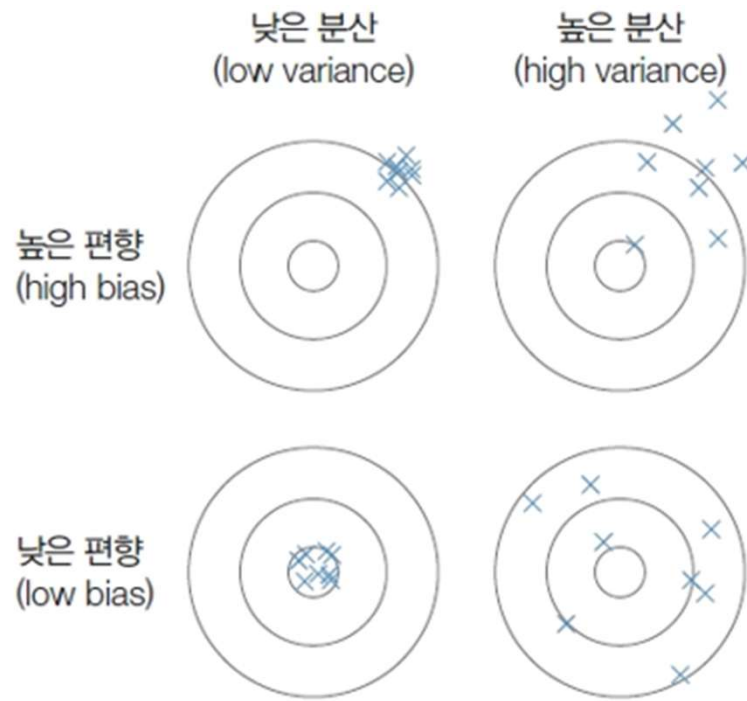


그림7. 편향-분산 트레이드오프

[TIP] 과대적합(overfitting) : 높은 분산 낮은 편향 상태로 함수가 훈련 데이터셋에만 맞음. 피쳐의 개수를 줄이거나 정규화하여 해결

[TIP] 과소적합(underfitting) : 낮은 분산 높은 편향 상태로 함수가 훈련 데이터셋과 테스트 데이터셋에 모두 맞지 않음. 피쳐를 추가하여 해결

선형회귀_과대적합과 정규화

- 과대적합이 발생할 때 경사하강법 루프가 진행될수록 학습 데이터셋에 대한 비용함수의 값은 줄어들이지만 테스트 데이터셋의 비용함수 값은 증가

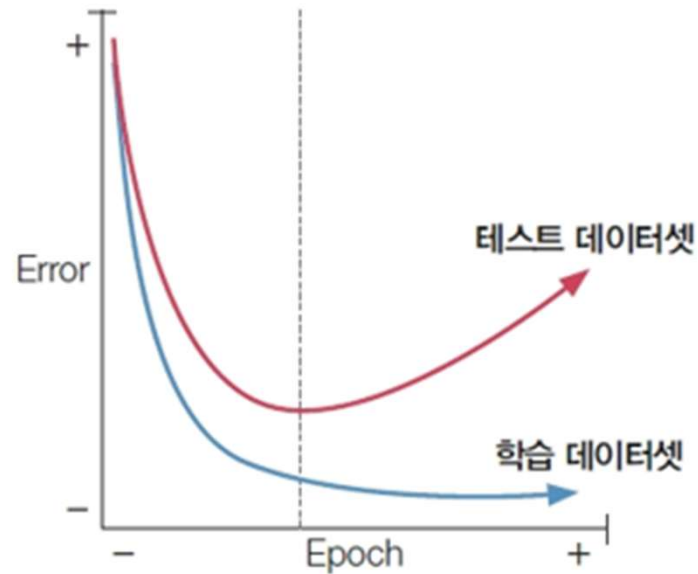


그림8. 과대적합이 발생할 때의 경사하강법

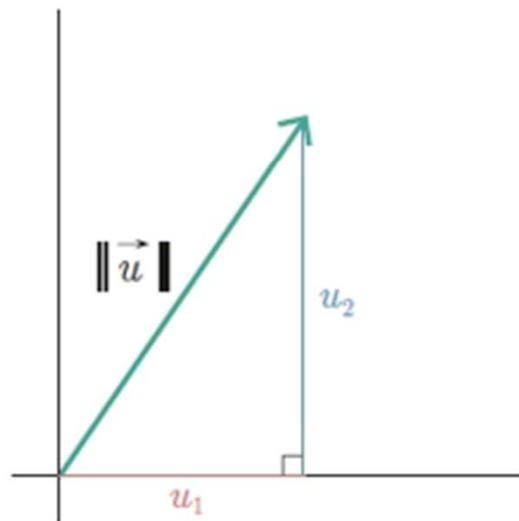
- 선형회귀 외에도 결정트리(decision tree)나 딥러닝처럼 연산에 루프가 필요한 모든 알고리즘에서 똑같이 발생

선형회귀_과대적합과 정규화

- 오컴의 면도날 원리 : '보다 적은 수의 논리로 설명이 가능한 경우, 많은 수의 논리를 세우지 않는다'
 - '경제성의 원리' 또는 '단순성의 원리'
 - 머신러닝에서는 너무 많은 피처를 사용하지 않는 것
- 선형회귀에서 과대적합 해결책
 - 더 많은 데이터 활용하기 : 오류가 없고, 분포가 다양한 데이터를 많이 확보
 - 피처의 개수 줄이기 : 필요한 피처만 잘 찾아 사용
 - 적절한 매개변수 선정하기 : SGD의 학습률이나 루프의 횟수처럼 적절한 하이퍼 매개변수를 선정
 - 정규화 적용하기 : 데이터 편향성에 따라 필요 이상으로 증가한 피처의 가중치 값을 적절히 줄이는 규제 수식을 추가

2. L2 정규화 : 리지 회귀

- 리지 회귀(ridge regression) : L2 정규화(L2 regularization)라고 부름
- 놈(norm) : 좌표평면의 원점에서 점까지의 거리를 나타내어 벡터의 크기를 측정하는 기법



$$L_{2\text{norm}} = \|x\|_2 = \sqrt{(\sum_i^n |x|^2)} = (\sum_i^n |x_i|^2)^{\frac{1}{2}}$$

$$\text{if } x = (x_1, x_2, \dots, x_n)$$

그림9. 놈(norm)의 이해

선형회귀_과대적합과 정규화

- x 는 하나의 벡터
- L2 놈(L2 norm) : 벡터 각 원소들의 제곱합에 제곱근을 취함
- 리지 회귀는 L2 놈을 선형회귀의 비용함수 수식에 적용

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^n \theta_j^2$$

- 뒷부분에 새로 붙인 수식은 페널티텀(penalty term)으로, 모델의 가중치 값들의 제곱의 합
 - 가중치 값이 조금이라도 커질 때 비용함수에 매우 큰 영향
 - λ 가 클수록 전체 페널티텀의 값이 커져 θ 값이 조절됨
 - λ 는 사람이 직접 값을 입력하는 하이퍼 매개변수

선형회귀_과대적합과 정규화

- 리지 회귀 수식을 미분하면 j 의 값이 1 이상일 때 페널티가 적용됨

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \left[\left(\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{m} \theta_j \right] j \in \{1, 2, \dots, n\}$$

$$\theta_j := \theta_j \left(1 - \alpha \frac{\lambda}{m} \right) - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

2. L1 정규화 : 라쏘 회귀

- 라쏘 회귀(lasso regression) L1 정규화(L1 regularization)라고 부름
- 가중치에 페널티를 추가하는데, 기존 수식에다 L1 놈 페널티를 추가하여 계산

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^n |\theta_j|$$

- L1 놈(L1 norm) : 절대값을 사용하여 거리를 측정

$$\|x\|_1 := \sum_{i=1}^n |x_i|$$

선형회귀_과대적합과 정규화

- L1 정규화와 L2 정규화가 실제 적용되는 과정

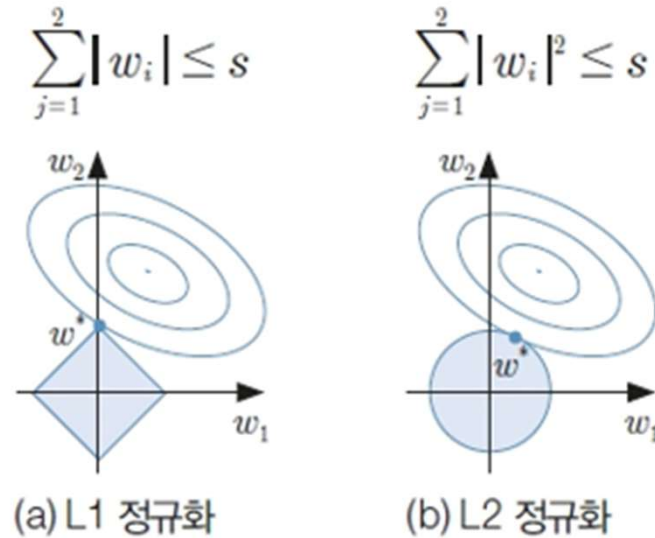


그림10. L1 정규화와 L2 정규화의 실제 적용 과정

- 타원은 두 개의 가중치 값의 최적점과 그 가중치 값으로 생기는 비용함수의 공통 범위
- 아래 마름모나 원은 w 가 가질 수 있는 범위이고 타원과 만나는 점이 바로 사용가능한 가중치 값

선형회귀_과대적합과 정규화

- L1정규화는 직선과 타원 만나는 점이 양쪽 끝에 생성됨
 - 극단적인 값이 생성되어 다른 가중치 값이 선택되지 않는 현상이 발생할 수 있음
 - 사용해야 하는 피쳐와 사용하지 않아도 되는 피쳐를 선택하여 사용하도록 지원
- L2 정규화는 원과 타원이 만나는 점이 많아져서 비교적 쉽게 연산되어 계산 효율(computational efficiency) 확보
 - 한 점에서 만나기 때문에 하나의 해답만 제공

선형회귀_과대적합과 정규화

5-4. lasso_ridge.py

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.linear_model import Ridge, Lasso
5 from sklearn.datasets import make_regression
6
7 # 1. 데이터 생성 (변수 100개 중 진짜 중요한 건 10개뿐인 상황)
8 X, y = make_regression(n_samples=50, n_features=100, n_informative=10, noise=1, random_state=42)
9
10 # 2. L1(Lasso)과 L2(Ridge) 학습
11 lasso = Lasso(alpha=1.0).fit(X, y)
12 ridge = Ridge(alpha=1.0).fit(X, y)
13
14 # [파트 1] 결과 수치 출력 (콘솔)
15 print("="*60)
16 print("== [가중치 값 비교 (상위 10개)] ==")
17 print("="*60)
18 df_comp = pd.DataFrame({
19     'L2 (Ridge) 가중치': ridge.coef_,
20     'L1 (Lasso) 가중치': lasso.coef_
21 })
22 print(df_comp.head(10)) # 상위 10개만 출력
23 print("\n" + "="*60)
24 # 결정적 차이: 가중치가 '정확히 0'인 변수의 개수 세기
25 print(f"L2 (Ridge)가 0으로 만든 변수 개수: {np.sum(ridge.coef_ == 0)}개 / 100개")
26 print(f"L1 (Lasso)가 0으로 만든 변수 개수: {np.sum(lasso.coef_ == 0)}개 / 100개")
27 print("="*60)
28
29 # [파트 2] 결과 시각화 (그래프)
30 plt.figure(figsize=(15, 6))
31
32 # 왼쪽: Ridge (가중치가 뺄림)
33 plt.subplot(*args: 1, 2, 1)
34 # 가중치 분포를 막대 그래프(Stem plot)로 표현
35 plt.stem(*args: ridge.coef_, markerfmt=' ', basefmt="k-")
36 plt.title(*args: "L2 (Ridge) Coefficients\n(Many small non-zero values)", fontsize=14)
37 plt.xlabel("Feature Index (0~99)")
38 plt.ylabel("Coefficient Value")
39 plt.ylim(*args: -100, 100) # 스케일 통일
40 plt.grid(*args: visible: True, alpha=0.3)
41
42 # 오른쪽: Lasso (대부분이 0이라 텅 비어 있음)
43 plt.subplot(*args: 1, 2, 2)
44 plt.stem(*args: lasso.coef_, markerfmt=' ', basefmt="k-")
45 plt.title(*args: "L1 (Lasso) Coefficients\n(Sparse: Most are exactly 0)", fontsize=14)
46 plt.xlabel("Feature Index (0~99)")
47 plt.ylabel("Coefficient Value")
48 plt.ylim(*args: -100, 100) # 스케일 통일
49 plt.grid(*args: visible: True, alpha=0.3)
50 plt.tight_layout()
51 plt.show()
```

Time series data를 활용해서 데이터 전처리 - 지도학습 - 성능평가를 진행.

- 데이터 전처리, 비교 모델, 성능평가 해석을 중점으로 설명

- PPT 구성을 자유롭게 하되 배운 이론을 중심으로 10분 이내 발표